

LAPORAN TUGAS BESAR

MATA KULIAH PEMROGRAMAN BERBASIS OBJEK

Disusun oleh:

1124013 - Michelle Daniella Alexandra

1124028 - Raffael David Yauson

1124048 - Fahad Wibawa Sukendar

1. Pendahuluan

1.1 Latar Belakang

Tugas besar ini dibuat untuk menerapkan konsep-konsep Pemrograman Berbasis Objek (PBO) yang telah dipelajari sepanjang perkuliahan ke dalam sebuah studi kasus yang nyata dan utuh, yaitu aplikasi Kiosk pemesanan makanan bergaya McDonald's. Aplikasi dibangun sepenuhnya menggunakan Java dan Java Swing, dengan struktur kode yang mengikuti pola Model-View-Controller (MVC) serta menerapkan konsep encapsulation, inheritance, polymorphism, abstraction, interface, generics, exception handling, anonymous class, hingga penerapan sebagian prinsip SOLID dan design pattern.

1.2 Deskripsi Aplikasi

Aplikasi terdiri dari dua sisi tampilan utama:

- Customer UI — dapat langsung digunakan tanpa login (mode kiosk), menampilkan kategori dan daftar menu, keranjang belanja, serta alur pembayaran (mock) dengan pilihan tunai, kartu debit/kredit, dan QRIS.
- Admin UI — mengharuskan login terlebih dahulu, digunakan untuk mengelola kategori, mengelola menu (tambah/ubah/hapus beserta gambar), dan mengelola status pesanan dari "Sudah Dibayar" menjadi "Selesai".

Seluruh data (kategori, menu, akun admin, dan pesanan) disimpan pada struktur data in-memory (ArrayList) yang diisi data contoh (dummy data) secara otomatis setiap aplikasi dijalankan, sehingga aplikasi dapat langsung dicoba tanpa perlu instalasi database eksternal.

2. Arsitektur Aplikasi

Aplikasi menggunakan pola arsitektur MVC (Model-View-Controller) yang dipadukan dengan pola DAO (Data Access Object). Alur data secara umum digambarkan sebagai berikut:

View → Controller → DAO (interface) → DAO (implementasi) → DataStore (in-memory)

↑
Model (POJO) + SessionManager (Singleton)

Layer	Package	Contoh Kelas	Tanggung Jawab
Model	tubes.model	User, Admin, Customer, Menu, Kategori, Order, OrderItem, Role, Status, PaymentMethod	Merepresentasikan data/objek bisnis (POJO)

Layer	Package	Contoh Kelas	Tanggung Jawab
View	tubes.view.*	LoginView, CustomerDashboard, MenuBrowseView, CartView, PaymentView, AdminDashboard, dll.	Tampilan antarmuka (Java Swing)
Controller	tubes.controller	AuthController, MenuController, KategoriController, OrderController, AdminController	Logika bisnis & validasi, penghubung View-DAO
DAO	tubes.dao / tubes.dao.interfaces	IMenuDAO/MenuDAO, IKategoriDAO/KategoriDAO, IOrderDAO/OrderDAO	Akses & manipulasi data
Data Store	tubes.database	DataStore (Singleton)	Penyimpanan data in-memory + seed data
Utility	tubes.util	SessionManager, AppColors, CurrencyUtil, ImageUtil, WrapLayout	Fungsi pendukung lintas layer

Tabel 1. Pembagian layer arsitektur MVC + DAO pada aplikasi.

3. Implementasi Konsep Pemrograman Berbasis Objek

Bagian ini memetakan setiap konsep PBO yang diajarkan pada tiap pertemuan perkuliahan terhadap implementasi nyatanya di dalam kode program.

Pertemuan 2: Class, Attribute, dan Encapsulation

Setiap class model membungkus (encapsulate) atributnya sebagai private, dan hanya dapat diakses/diubah lewat method getter dan setter publik. Ini mencegah data dimanipulasi langsung dari luar class.

File / Class	Atribut Private	Getter/Setter
model/User.java	id, username, password, nama, role	getId(), getUsername(), setPassword(), dll.
model/Menu.java	id, namaMenu, kategori, harga, stok, gambar, tersedia	getHarga(), setStok(), isTersedia(), dll.
model/Order.java	id, customer, status, total, pajak, items, bayar, kembalian	getGrandTotal(), setStatus(), dll.
model/Kategori.java	id, nama	getNama(), setNama()
model/Customer.java	cart (List<OrderItem>)	getCart(), getCartTotal(), addToCart()

Tabel 2. Contoh penerapan encapsulation pada class model.

Pertemuan 2: Constructor, Constructor Overloading, dan Keyword this

Beberapa class menyediakan lebih dari satu constructor (constructor overloading) untuk kebutuhan yang berbeda, misalnya constructor lengkap dengan id (dipakai saat memuat data yang sudah ada) dan constructor tanpa id (dipakai saat membuat data baru, id akan diberikan otomatis oleh DataStore). Keyword this dipakai

baik untuk membedakan atribut dengan parameter, maupun untuk memanggil constructor lain dalam class yang sama.

File / Class	Constructor yang Tersedia	Keterangan
model/User.java	User(id, username, password, nama, role) User(username, password, nama, role)	Constructor kedua memanggil this(0, ...)
model/Menu.java	Menu(id, nama, kategori, harga, stok, gambar, tersedia) Menu(nama, kategori, harga, stok, gambar)	Overloading untuk data baru vs data lama
model/Order.java	Order(customer, userId) Order(id, customer, userId, status, total, catatan, tanggal)	Overloading untuk order baru vs rekonstruksi
model/OrderItem.java	OrderItem(menu, qty) OrderItem(id, orderId, menu, qty, harga, subtotal)	Overloading serupa Order.java
model/Kategori.java	Kategori(id, nama) Kategori(nama)	Constructor kedua memanggil this(0, nama)

Tabel 3. Contoh constructor overloading pada class model.

Pertemuan 3: Inheritance (Pewarisan)

Class Admin dan Customer mewarisi (extends) seluruh atribut dan method dari class abstrak User, lalu memanfaatkan super(...) untuk meneruskan data ke constructor induk. Beberapa komponen tampilan (VirtualKeyboard, RoundedButton, StatusBadge, dll.) juga mewarisi class bawaan Java Swing untuk menambah perilaku baru.

Subclass	Superclass	Pemanfaatan super()/override
model/Admin.java	model/User.java (abstract)	super(id, username, password, nama, Role.ADMIN)
model/Customer.java	model/User.java (abstract)	super(id, username, password, nama, Role.CUSTOMER)
view/components/VirtualKeyboard.java	javax.swing.JWindow	Menambah panel tombol keyboard on-screen
view/components/RoundedButton.java	javax.swing.JButton	Override paintComponent() untuk sudut membulat
view/components/StatusBadge.java	javax.swing.JLabel	Override paintComponent() untuk badge berwarna
util/WrapLayout.java	java.awt.FlowLayout	Override perhitungan layout agar responsif

Tabel 4. Contoh penerapan inheritance pada class model dan komponen UI.

Pertemuan 3 & 5: Abstract Class, Abstract Method, dan Polymorphism

Class User dideklarasikan abstract dan memiliki dua abstract method, yaitu showDashboard() dan getRoleDescription(). Kedua method ini diimplementasikan berbeda oleh Admin dan Customer (polymorphism / method overriding), sehingga pemanggilan method yang sama pada objek User menghasilkan perilaku berbeda tergantung objek sebenarnya (Admin atau Customer).

Class	Method Abstrak	Implementasi (Override)
model/User.java (abstract)	abstract void showDashboard()	—
model/Admin.java	showDashboard()	Mencetak info dashboard Admin
model/Customer.java	showDashboard()	Mencetak info dashboard Customer/Kiosk
model/User.java (abstract)	abstract String getRoleDescription()	—
model/Admin.java	getRoleDescription()	"Administrator — Akses penuh ke semua modul"
model/Customer.java	getRoleDescription()	"Customer — Memesan menu melalui kiosk"

Tabel 5. Contoh abstract method yang di-override (polymorphism).

Pertemuan 5: Interface dan Abstraction

Layer DAO menggunakan interface (IMenuDAO, IKategoriDAO, IOrderDAO) sebagai kontrak, sehingga Controller hanya bergantung pada abstraksi (interface), bukan pada implementasi konkret. Hal ini memudahkan penggantian implementasi penyimpanan data (misalnya dari in-memory ke database) tanpa mengubah kode Controller maupun View. Selain itu, komponen MenuCard mendefinisikan interface fungsional sendiri (MenuCardListener) sebagai bentuk callback.

Interface	Diimplementasikan Oleh	Digunakan Oleh
dao/interfaces/IMenuDAO.java	dao/MenuDAO.java	controller/MenuController.java
dao/interfaces/IKategoriDAO.java	dao/KategoriDAO.java	controller/KategoriController.java
dao/interfaces/IOrderDAO.java	dao/OrderDAO.java	controller/OrderController.java
view/components/MenuCard.java (interface MenuCardListener)	Lambda pada MenuBrowseView.java	MenuCard.java (callback saat tombol "+ Keranjang" ditekan)

Tabel 6. Contoh interface sebagai kontrak/abstraksi antar layer.

Pertemuan 5: MVC (Model-View-Controller)

Pemisahan tanggung jawab kode dilakukan secara konsisten: Model hanya berisi data (POJO), View hanya berisi tampilan Swing tanpa logika bisnis, dan Controller menjadi penghubung yang memvalidasi input serta memanggil DAO.

Komponen	Contoh Kelas	Peran
Model	Menu.java, Order.java, Kategori.java	Menyimpan struktur data murni, tanpa logika UI
View	MenuBrowseView.java, CartView.java, PaymentView.java	Menampilkan data & menangkap input pengguna
Controller	MenuController.java, OrderController.java	Validasi input, memanggil DAO, mengembalikan hasil/pesan error ke View

Tabel 7. Pemetaan komponen MVC pada aplikasi.

Materi Tambahan: Anonymous Inner Class dan Lambda Expression

Konsep anonymous inner class (seperti pada contoh materi Anonymous.java dengan Comparator) banyak digunakan pada penanganan event Java Swing (MouseListener, WindowListener, FocusListener). Untuk interface fungsional (hanya satu abstract method), kode ini juga memanfaatkan lambda expression sebagai bentuk penulisan yang lebih ringkas dari anonymous class.

File	Bentuk	Contoh Penggunaan
view/components/MenuCard.java	Anonymous class (MouseAdapter)	addMouseListener(new MouseAdapter() {...}) untuk efek hover kartu menu
view/LoginView.java	Anonymous class (WindowAdapter, FocusAdapter)	Menangani event tutup jendela dan fokus pada text field
view/components/VirtualKeyboard.java	Anonymous class (MouseAdapter)	Efek hover pada tombol-tombol keyboard on-screen
view/customer/MenuBrowseView.java	Lambda (MenuCardListener)	card.setListener((menu, qty) -> addToCart(menu, qty))
model/Customer.java	Lambda (Predicate)	cart.removeIf(item -> {...}) untuk menghapus item keranjang
view/customer/CartView.java	Lambda (ActionListener)	btnCheckout.addActionListener(e -> goToPayment())

Tabel 8. Contoh anonymous inner class dan lambda expression.

Pertemuan 6: Generics

Generics diterapkan secara konsisten melalui parameterized type dari Java Collections (List<T>) di seluruh layer Model, DAO, dan Controller, sehingga tipe data yang disimpan dalam koleksi terjamin aman secara tipe (type-safe) pada saat kompilasi.

File / Class	Penggunaan Generics	Manfaat
dao/interfaces/IMenuDAO.java	List<Menu> getAll();	Menjamin isi List hanya bertipe Menu
dao/interfaces/IOrderDAO.java	List<Order> getByStatus(Status s);	Type-safety pada hasil query order
model/Order.java	List<OrderItem> items;	Koleksi item pesanan bertipe tetap
model/Customer.java	List<OrderItem> cart;	Koleksi keranjang belanja bertipe tetap
view/customer/*, view/admin/*	SwingWorker<List<Menu>, Void>	Generics dua parameter: tipe hasil & tipe progres

Tabel 9. Contoh penerapan generics (parameterized type).

Catatan: berbeda dengan contoh bounded generics pada materi (Box<T extends Doubleable>, wildcard <?> pada GenericsBound.java), aplikasi ini lebih banyak memanfaatkan generics sederhana melalui Java Collections Framework karena kebutuhan aplikasi kiosk tidak memerlukan struktur data generik kustom.

Pertemuan 7: Exception Handling

Penanganan error dilakukan dengan dua pendekatan yang saling melengkapi: (1) try-catch untuk operasi yang benar-benar dapat melempar exception bawaan Java (parsing angka, I/O file gambar), dan (2) pola return message (mirip contoh materi Teori9d2Return.java) pada method validasi Controller, yang mengembalikan String pesan error (atau null jika sukses) alih-alih melempar custom exception, supaya pesan kesalahan dapat langsung ditampilkan ke pengguna kios melalui dialog.

File / Method	Jenis Exception	Penanganan
controller/MenuController.java validateInput()	NumberFormatException	catch → dikembalikan sebagai pesan error String ke View
controller/MenuController.java copyGambar()	IOException	catch → log error, method mengembalikan null dengan aman
util/ImageUtil.java loadMenuImage()	Exception (umum)	catch → fallback ke gambar placeholder, aplikasi tidak crash
main/Main.java	Exception (umum)	try-catch pada saat inisialisasi Look and Feel dan Splash Screen
controller/*Controller.java (semua method validasi)	Bukan exception, tapi return String	Return null = sukses, return String = pesan error (pendekatan defensif)

Tabel 10. Contoh penanganan exception dan validasi error.

Pertemuan 8: SOLID Principles dan Design Pattern

Beberapa prinsip SOLID dan design pattern klasik diterapkan pada struktur kode, meskipun tidak seluruh prinsip SOLID diterapkan secara ketat mengingat skala aplikasi.

Prinsip / Pattern	Implementasi	Lokasi
Singleton Pattern	Constructor private + instance static tunggal	database/DataStore.java, util/SessionManager.java
DAO Pattern	Pemisahan logika akses data dari logika bisnis	package tubes.dao & tubes.dao.interfaces
Single Responsibility (SRP)	Tiap class punya satu tanggung jawab (Model/View/Controller/DAO)	Seluruh package tubes.*
Dependency Inversion (DIP)	Controller bergantung ke interface DAO, bukan implementasi	controller/*.java → dao/interfaces/*.java
Open/Closed (OCP) — sebagian	Menambah implementasi DAO baru tidak mengubah kode Controller	dao/interfaces/*.java

Tabel 11. Contoh penerapan SOLID principle dan design pattern.

Pertemuan 9: Konsep ORM / Data Persistence (Adaptasi)

Pada materi, ORM dipelajari sebagai cara memetakan objek Java ke tabel database (mapping). Pada aplikasi kios ini, pemetaan objek-ke-penyimpanan tetap diterapkan secara konseptual, namun media penyimpanannya diganti dari database relasional (MySQL/JDBC) menjadi struktur in-memory (ArrayList pada DataStore), supaya aplikasi kios dapat langsung dijalankan tanpa instalasi database eksternal. Pola DAO tetap dipertahankan sebagai lapisan abstraksi data, sehingga jika suatu saat dikembangkan kembali ke database sungguhan, hanya kelas implementasi DAO yang perlu diganti.

Konsep ORM	Adaptasi pada Aplikasi	Lokasi
Tabel database	List<T> pada DataStore (mis. List<Menu>, List<Order>)	database/DataStore.java
Primary key / auto increment	AtomicInteger sebagai penghasil ID unik	database/DataStore.java (menuIdSeq, orderIdSeq, dst.)
Repository/DAO ke database	Repository/DAO ke koleksi in-memory	dao/MenuDAO.java, dao/OrderDAO.java, dao/KategoriDAO.java
Seed data / migrasi awal	Method seed() yang mengisi data contoh saat aplikasi start	DataStore.seed()

Tabel 12. Adaptasi konsep ORM menjadi data persistence in-memory.

4. Kesimpulan

Aplikasi Kiosk McDonald's ini berhasil mengimplementasikan hampir seluruh konsep Pemrograman Berbasis Objek yang diajarkan selama satu semester, mulai dari konsep dasar (class, encapsulation, constructor overloading), konsep pewarisan (inheritance, abstract class, polymorphism), konsep abstraksi (interface, MVC), konsep lanjutan (generics, anonymous class, lambda expression, exception handling), hingga penerapan sebagian prinsip SOLID dan design pattern (Singleton, DAO). Konsep ORM diadaptasi menjadi mekanisme penyimpanan data in-memory agar aplikasi kiosk dapat langsung dijalankan tanpa proses instalasi database tambahan, tanpa mengorbankan struktur arsitektur berlapis (MVC + DAO) yang sudah diterapkan.